STEP 2: 4-BIT OPERATIONS

BLUE GIRAFFES

Connor Nauert, Daniel Okoh, James Webb

CS.3339 Computer Architecture

Texas State University

October 20, 2026

1 Introduction

In this project, we are designing a small computer from scratch by using Verilog for hardware programming. We will also be using GTKWave to analyze test case waveforms and display the inputs and outputs of the gates.

Step 2 involves building upon small 1-bit circuits made in step 1 by increasing input size to 4 bits. These new gates will also be supplemented with examples of adders, subtractors, multipliers, and dividers commonly used in ALUs.

2 Verilog Code

In this section we will show and explain our implementations of the circuits and their testbenches.

2.1 And Circuit

The And circuit will take 2 4-bit inputs A and B, and output 1 4-bit output X. For each bit of A and B, it will output a 1 for 1 and 1, and a 0 otherwise (0101, 0011 = 0001).

```
/*
 * File: and4.v
 * Description: bitwise AND of two 4-bit inputs
 * Inputs:  A[3:0], B[3:0]
 * Outputs: X[3:0]
 * Authors: Blue Giraffes
 * Course:  CS 3339 Computer Architecture, Texas State University
 * Project: Integer ALU, Step 2
 */

module and4 (A, B, X);   // module name and port list

    input [3:0] A;    // first 4-bit operand
    input [3:0] B;    // second 4-bit operand
    output [3:0] X;    // 4-bit result

    assign X = A & B;   // continuous assignment of the result

endmodule   // end of and4
```

2.2 Nand Circuit

The Nand circuit is composed of a Not gate and an And gate. Two 4-bit inputs, A and B, are entered into an And gate. The output of the And gate is then inverted and assigned to X. This means that if the same bit of A and B are both 1, the And gate will invert the combined 1 into a 0. The other three states will output a 1 (0101, 0011 = 1110).

```
/*
 * File: nand4.v
 * Description: bitwise NAND of two 4-bit inputs
 * Inputs: A[3:0], B[3:0]
 * Outputs: X[3:0]
 * Authors: Blue Giraffes
 * Course: CS 3339 Computer Architecture, Texas State University
 * Project: Integer ALU, Step 2
 */

module nand4 (A, B, X);  // module name and port list

    input [3:0] A;  // first 4-bit operand
    input [3:0] B;  // second 4-bit operand
    output [3:0] X;  // 4-bit result

    assign X = ~(A & B);   // continuous assignment of the result

endmodule
```

The Nand circuit requires us to test for four seperate states. A and B are taken in as inputs and set to a pattern. We then wait ten nanoseconds, and repeat for all four states.

2.3 Or Circuit

The Or circuit takes 2 4-bit inputs A and B and will output for each bit a 0 on 0 and 0, and a 1 otherwise into a 4-bit output X (0101, 0011 = 0111).

```
/*
 * File: or4.v
 * Description: bitwise OR of two 4-bit inputs
 * Inputs:  A[3:0], B[3:0]
 * Outputs: X[3:0]
 * Authors: Blue Giraffes
```

```
 * Course: CS 3339 Computer Architecture, Texas State University
 * Project: Integer ALU, Step 2
 */

module or4 (A, B, X); // module name and port list

    input  [3:0] A;  // first 4-bit operand
    input  [3:0] B;  // second 4-bit operand
    output [3:0] X;  // 4-bit result

    assign X = A | B;   // continuous assignment of the result

endmodule
```

2.4 Nor Circuit

The Nor circuit is similar to the Nand circuit in that it is the inverse of another gate. The Or gate will take two 4-bit inputs, A and B. The Not gate will switch each bit output by the Or gate, which is then wired to X. If both A and B are 0, then X should be 1. Otherwise, X is 0 (0101, 0011 = 1000).

```
/*
 * File: nor4.v
 * Description: bitwise NOR of two 4-bit inputs
 * Inputs: A[3:0], B[3:0]
 * Outputs: X[3:0]
 * Authors: Blue Giraffes
 * Course: CS 3339 Computer Architecture, Texas State University
 * Project: Integer ALU, Step 2
 */

module nor4 (A, B, X);   // module name and port list

    input  [3:0] A;  // first 4-bit operand
    input  [3:0] B;  // second 4-bit operand
    output [3:0] X;  // 4-bit result

    assign X = ~(A | B);   // continuous assignment of the result

endmodule
```

2.5 Xor Circuit

The Xor circuit will take 2 4-bit inputs A and B and will output for each bit a 0 on 0 and 0 or 1 and 1, and a 1 otherwise into a 4-bit output X (0101, 0011 = 0110).

```
/*
 * File: xor4.v
 * Description: bitwise XOR of two 4-bit inputs
 * Inputs: A[3:0], B[3:0]
 * Outputs: X[3:0]
 * Authors: Blue Giraffes
 * Course: CS 3339 Computer Architecture, Texas State University
 * Project: Integer ALU, Step 2
 */

module xor4 (A, B, X);  // module name and port list

    input  [3:0] A;  // first 4-bit operand
    input  [3:0] B;  // second 4-bit operand
    output [3:0] X;  // 4-bit result

    assign X = A ^ B;  // continuous assignment of the result

endmodule
```

2.6 Xnor Circuit

The Xnor circuit is the inverse of Xor and will take 2 4-bit inputs A and B and will output for each bit a 1 on 0 and 0 or 1 and 1, and a 0 otherwise into a 4-bit output X (0101, 0011 = 1001).

```
/*
 * File: xnor4.v
 * Description: bitwise XNOR of two 4-bit inputs
 * Inputs: A[3:0], B[3:0]
 * Outputs: X[3:0]
 * Authors: Blue Giraffes
 * Course: CS 3339 Computer Architecture, Texas State University
 * Project: Integer ALU, Step 2
 */

```

```
module xnor4 (A, B, X); // module name and port list

    input  [3:0] A;  // first 4-bit operand
    input  [3:0] B;  // second 4-bit operand
    output [3:0] X;  // 4-bit result

    assign X = ~(A ^ B);  // continuous assignment of the result

endmodule
```

2.7 Not Circuit

The Not circuit takes one 4-bit input A and flips each bit from 1 to 0 or from 0 to 1 and then outputs 1 4-bit output X (0101 = 1010).

```
/*
 * File: not4.v
 * Description: bitwise NOT of a 4-bit input
 * Inputs:  A[3:0]
 * Outputs: X[3:0]
 * Authors: Blue Giraffes
 * Course: CS 3339 Computer Architecture, Texas State University
 * Project: Integer ALU, Step 2
 */

module not4 (A, X);  // module name and port list

    input  [3:0] A;   // 4-bit operand
    output [3:0] X;   // 4-bit result

    assign X = ~A;  // invert every bit of A

endmodule
```

Testing the Not circuit involves creating a register A and measuring an output X. There are two possible states to be tested: when a bit of A is 0 and when a bit of A is 1. The pound symbol in the snippet below calls for the computer to wait for some amount of nanoseconds, which allows us to observe how the gate acts over time.

2.8 2×4 bit Arithmetic Shifter

The Arithmetic shifter circuit takes 2 inputs A and B, and then shifts A according to B. The left-most bit of B determines if it shifts left or right, with 0 being left and 1 being right. The right-most bit determines what the shift will fill with (0 with 0, 1 with 1). The middle 2 bits determine the amount it shifts (01 shift 1, 11 shift 3).

```
/*
 * File: shift4.v
 * Description: 4-bit arithmetic shifter with fill bit and overflow output
 * Inputs: A[3:0] = value to shift
 *         B[3] = direction (0 left, 1 right), B[2:1] = shift amount, B[0] = fill bit
 * Outputs: X[3:0] = shifted value, Y[3:0] = bits shifted out of A
 * Authors: Blue Giraffes
 * Course:  CS 3339 Computer Architecture, Texas State University
 * Project: Integer ALU, Step 2
 */

module shift4 (A, B, X, Y);   // module name and port list

    input  [3:0] A;  // value to be shifted
    input  [3:0] B;  // control word B3 B2 B1 B0
    output reg [3:0] X;  // shifted result
    output reg [3:0] Y;  // overflow (bits shifted out)

    reg [7:0] wide;  // 8-bit workspace {Y, X} so shifted-out bits are kept
    reg [1:0] n;  // shift amount taken from B[2:1]
    reg [3:0] mask;  // marks the positions that must be filled

    always @(*) begin  // re-evaluate whenever an input changes
        n = B[2:1];  // shift amount is 0 to 3

        if (B[3] == 1'b0) begin  // B3 = 0 means shift left
            wide = {4'b0000, A} << n;  // A sits in the low half, shift toward the high half
            mask = (4'b0001 << n) - 4'b0001;  // n ones in the low bits (the vacated spots)
```

```
29              X = wide[3:0] | (B[0] ? mask : 4'b0000);  // low half is
    X, add fill ones if B0 = 1
30              Y = wide[7:4];  // high half holds the bits that left A
31          end else begin   // B3 = 1 means shift right
32              wide = {A, 4'b0000} >> n;  // A sits in the high half,
    shift toward the low half
33              mask = ~(4'b1111 >> n);  // n ones in the top bits (the
    vacated spots)
34              X = wide[7:4] | (B[0] ? mask : 4'b0000);  // high half
    is X, add fill ones if B0 = 1
35              Y = wide[3:0];  // low half holds the bits that left A
36          end   // end direction select
37      end   // end always
38
39  endmodule   // end of shift4
```

2.9 Addition Circuit

The Addition circuit takes 2 inputs A and B, does A+B, and then outputs one 4-bit output Sum (0101, 0011 = 1000).

```
1  /*
2   * File: add4.v
3   * Description: 4-bit adder with carry in and carry out
4   * Inputs:  A[3:0], B[3:0], Cin
5   * Outputs: Sum[3:0], Cout
6   * Authors: Blue Giraffes
7   * Course:  CS 3339 Computer Architecture, Texas State University
8   * Project: Integer ALU, Step 2
9   */
10
11 module add4 (A, B, Cin, Sum, Cout);   // module name and port list
12
13      input [3:0] A;      // first operand
14      input [3:0] B;      // second operand
15      input Cin;    // carry in
16      output [3:0] Sum;     // 4-bit sum
17      output Cout;   // carry out
18
```

```
19      assign {Cout, Sum} = A + B + Cin;   // 5-bit result: top bit is
        the carry out
20
21  endmodule
```

2.10 Subtraction Circuit

The Subtraction circuit takes 2 inputs A and B, does A-B, and then outputs 1 4-bit 2s-complement number Diff, with the sign bit being outputted in Bout (0101, 0011 = 0010, 0).

```
1  /*
2   * File: sub4.v
3   * Description: 4-bit subtractor with borrow in and borrow out (Diff
        = A - B - Bin)
4   * Inputs:  A[3:0], B[3:0], Bin
5   * Outputs: Diff[3:0], Bout (1 when the result would be negative)
6   * Authors: Blue Giraffes
7   * Course: CS 3339 Computer Architecture, Texas State University
8   * Project: Integer ALU, Step 2
9   */
10
11 module sub4 (A, B, Bin, Diff, Bout);  // module name and port list
12
13     input [3:0] A;  // minuend
14     input [3:0] B;  // subtrahend
15     input Bin;   // borrow in
16     output [3:0] Diff;  // 4-bit difference
17     output Bout;  // borrow out
18
19     assign {Bout, Diff} = {1'b0, A} - {1'b0, B} - Bin;  // 5-bit
       subtract: top bit is the borrow
20
21 endmodule
```

2.11 Multiplication Circuit

The Multiplication circuit takes 2 4-bit outputs A and B, does A*B, and then outputs into 2 4-bit outputs X and Y, with Y being the top 4 bits, and X being the bottom 4 bits (0101, 0011 = 0000,1111).

```
/*
 * File: mul4.v
 * Description: 4-bit multiplier with an 8-bit product split across
   two 4-bit outputs
 * Inputs: A[3:0], B[3:0]
 * Outputs: X[3:0] = low half of A*B, Y[3:0] = high half of A*B
 * Authors: Blue Giraffes
 * Course: CS 3339 Computer Architecture, Texas State University
 * Project: Integer ALU, Step 2
 */

module mul4 (A, B, X, Y); // module name and port list

    input [3:0] A;  // multiplicand
    input [3:0] B;  // multiplier
    output [3:0] X;  // low 4 bits of the product
    output [3:0] Y;  // high 4 bits of the product

    wire [7:0] product; // full 8-bit product (4 bits x 4 bits needs
    8 bits)

    assign product = A * B;  // multiply the operands
    assign X = product[3:0];  // lower nibble
    assign Y = product[7:4];  // upper nibble

endmodule
```

2.12 Division Circuit

The Division Circuit takes 2 4-bit inputs A and B, does A/B, then outputs 2 4-bit outputs X and Y, with X being the quotient and Y being the remainder (1101, 0011 = 0100, 0001).

```
/*
 * File: div4.v
 * Description: 4-bit unsigned divider with quotient and remainder
   outputs
 * Inputs: A[3:0] = dividend, B[3:0] = divisor
 * Outputs: X[3:0] = quotient, Y[3:0] = remainder
 * Authors: Blue Giraffes
 * Course: CS 3339 Computer Architecture, Texas State University
```

```
 * Project: Integer ALU, Step 2
 */

module div4 (A, B, X, Y);   // module name and port list

    input [3:0] A;         // dividend
    input [3:0] B;         // divisor
    output reg [3:0] X;     // quotient
    output reg [3:0] Y;     // remainder

    always @(*) begin   // re-evaluate whenever an input changes
        if (B == 4'b0000) begin   // dividing by zero is undefined
            X = 4'b1111;   // flag the error with the largest
   quotient
            Y = A;   // pass the dividend through as the remainder
        end else begin   // normal division
            X = A / B;   // integer quotient
            Y = A % B;   // remainder
        end   // end divide-by-zero check
    end   // end always

endmodule   // end of div4
```

3 Waveform Tests

Now, we will showcase waveforms created by testing our twelve circuits. All waves were generated by vvp and rendered with GTKWave.

3.1 Not Circuit Waveform

In the first half of the waveform, our input (A) is 0, or low. This is inverted to 1 (high) in the output X. After 20ns, A is flipped to 1 and X changes accordingly to 0.

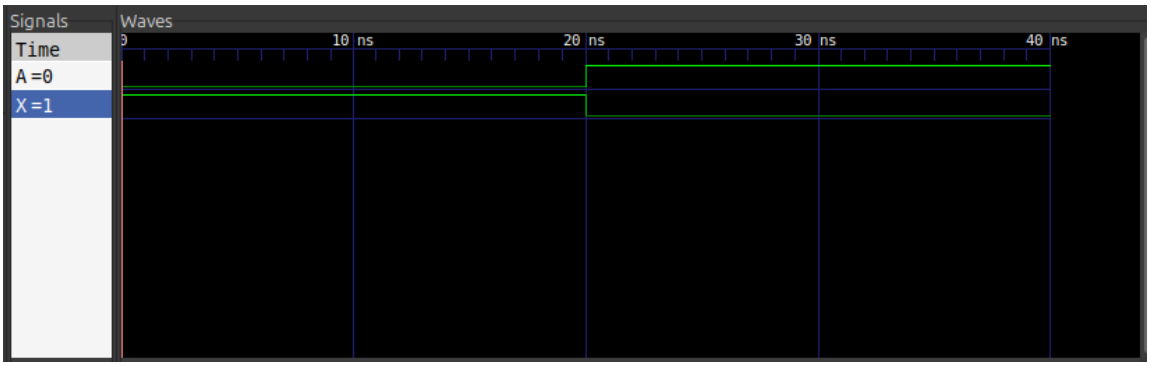


Figure 1: Not circuit waveform. There are two sections since there are two states.

3.2 Nand Circuit Waveform

A and B are initially set to 0, and our output X is correctly set to 1. X stays high through two other states until both A and B are set to 1 at 30ns. Since Nand is the inverse of And, X is then set to 0.

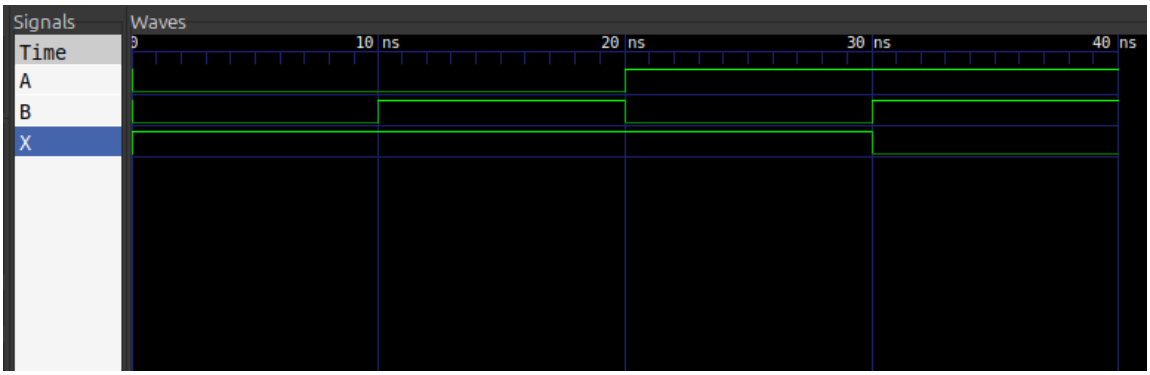


Figure 2: Nand circuit waveform. Notice we now have four states.

3.3 Nor Circuit Waveform

Again, A and B are set to 0 at the start of the test. X is high for 10ns since neither A nor B are high. For the remaining 30ns, at least one input is a 1 causing X to become a 0.

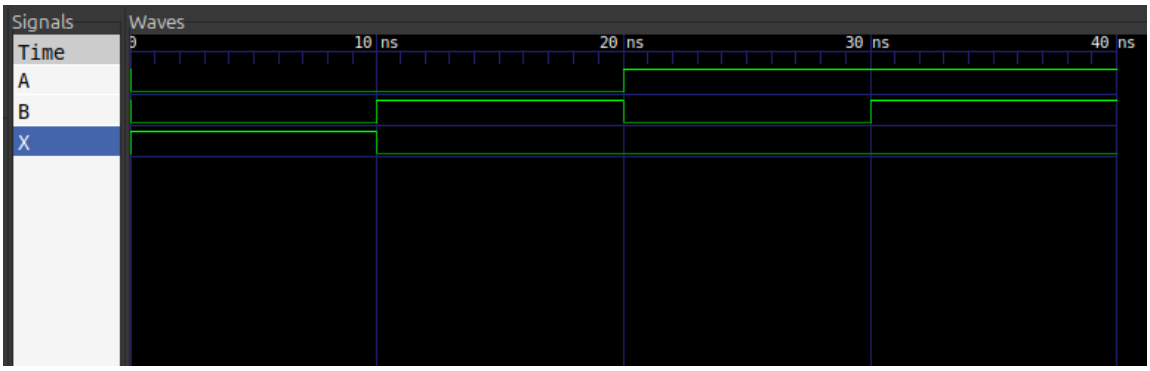


Figure 3: Nor circuit waveform.

3.4 1x4 Arithmetic Shifter Waveform

The Arithmetic shifter waveform looks quite complex, but each test case can be explained easily. There are two 4-bit arrays: one for an input and another for the shifted output. On the top, there is a direction register that shifts to the right when 0 and to the left when 1.

At the start, the input array reads *0110*, and the direction reads *0*. This means that we will shift the input to the right one bit. Sure enough, the output is correctly set to the shifted pattern of *0011*. When the direction bit flips to left after 10ns, the output becomes *1100*.

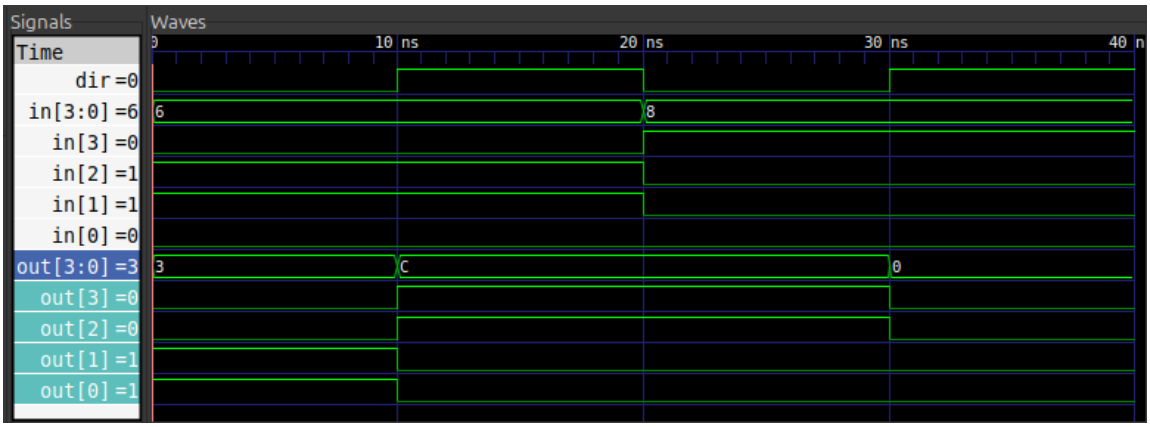


Figure 4: Arithmetic shifter waveforms, with a DIR register and 8 bits.

4 Conclusion

We have successfully created four 1-bit circuits in Verilog, tested them under specific inputs, and observed their behavior through waveforms in GTKWave.

We found that the most difficult part of this project was learning how Verilog source files were properly structured and compiled. Luckily, there was a plethora of online documentation and examples on the internet to learn from. LaTeX was helpful as the typesetting was done for us and we could focus on the content of the report. We used Git to collaborate and track changes. We trust that the next few project steps will go smoother considering everything we have learned.